

VTC ACADEMY

PROJECT REPORT
ENDLESS ZOMBIE

Survival Shooter / Action Roguelite

Information	Details
Semester	Project II
Class	GD1305
Group	The MoonStone
Instructor	Nguyen Dinh Cuong
Group Member	Vu Viet Tien
Engine	Unity 6000.3.10f1

This report follows the structure of the supplied Project II report form and is based on the current Endless Zombie source code, prefabs, and configuration assets.

TABLE OF CONTENTS

1. Project Overview	3
2. Project Scope	3
3. System Name	3
4. Deployment Environment	3
5. Development Tools	3
6. System Features	4
1. System Overview	5
2. Functional Requirements	5
3. Non-functional Requirements	5
4. Use Case Diagram	5
5. Use Case Descriptions	5
UC-01 - Start a Stage	5
UC-02 - Move and Auto Attack	6
UC-03 - Spawn an Enemy Wave	6
UC-04 - Level Up and Choose an Upgrade	6
UC-05 - Pause and Configure Settings	6
UC-06 - Complete a Stage	7
6. Activity Diagram	7
1. UI Design	8
2. Hybrid Architecture Diagram	8
3. Class / Component Diagram	8
4. Sequence Diagram	8
5. Gameplay Design	8
5.1. Wave and Spawn Pipeline	8
5.2. Weapons and Shotgun Spread	8
5.3. Enemy Visuals and Animation	9
5.4. State Machine	9
6. Data Model	9
Acceptance Criteria	10
Self-assessment	11
1. Prerequisites	12
2. Open the Project	12
3. Run in the Editor	12
4. Build for Windows	12

I. PROJECT INTRODUCTION

1. Project Overview

Endless Zombie is a top-down survival shooter in which the player controls an armed survivor inside an arena and attempts to survive increasingly difficult enemy waves. The combat loop combines movement, automatic target acquisition, weapon behavior, experience collection, level-up choices, and persistent upgrades purchased from the main menu.

The project uses a hybrid architecture. GameObject-oriented code handles input, menus, Canvas UI, audio, presentation, and game states. Unity Entities/DOTS handles wave scheduling, entity spawning, movement, projectiles, damage, combat events, and XP. Bridge systems synchronize Entity state with GameObject models, Animator controllers, and visual effects.

2. Project Scope

- A Canvas-based main menu for stage selection, loadout selection, meta upgrades, and settings.
- Player movement, automatic aiming, and several weapon profiles, including a multi-projectile Shotgun.
- Data-driven stage and wave spawning through StageConfig and EnemyCatalog assets.
- Regular zombie and DogMutant visual variants with movement-to-animation speed synchronization.
- A Canvas-based gameplay HUD for health, XP, level, wave status, currency, and weapon statistics.
- Running, paused, level-up, game-over, and win states.
- Local persistence for music, sound, vibration, gold, and meta progression.

3. System Name

Endless Zombie - Hybrid ECS Survival Shooter.

4. Deployment Environment

Item	Configuration
Target platform	Windows PC; responsive UI bootstrap is also available for mobile layouts
Engine	Unity 6000.3.10f1
Render pipeline	Universal Render Pipeline 17.3.0
Data-oriented stack	Unity Entities 1.4.2, Entities Graphics 1.4.15, Unity Physics 1.4.2
Language	C#
Input	Unity Input System 1.18.0

5. Development Tools

Tool	Purpose
Unity Editor	Scenes, prefabs, animation, physics, profiling, and builds
Visual Studio / Rider	C# development and debugging
Git	Version control
TextMesh Pro / UGUI	Canvas menus and HUD
ScriptableObject assets	Stage, enemy, weapon, visual, and upgrade configuration

6. System Features

The implemented feature set includes data-driven waves, prefab-based enemy entities, chase behavior, projectiles, damage and status effects, XP and level-up upgrades, weapon profiles, persistent progression, editable Canvas UI, audio controls, and game completion states.

II. SYSTEM REQUIREMENTS ANALYSIS

1. System Overview

The system is organized into four responsibility groups: Presentation, OOP Gameplay, ECS Simulation, and Configuration/Persistence. This division keeps menu and HUD layout editable in the Unity Inspector while moving high-volume gameplay data into cache-friendly ECS systems.

2. Functional Requirements

ID	Requirement	Priority
FR-01	Start a selected stage from the main menu and load GameScene.	Must
FR-02	Move the player and automatically acquire a valid target.	Must
FR-03	Spawn enemies by wave, prefab, schedule, and maximum-alive limit.	Must
FR-04	Process projectiles, collisions, damage, death, and XP.	Must
FR-05	Present and apply level-up upgrade choices.	Must
FR-06	Display an Inspector-editable Canvas HUD.	Must
FR-07	Pause, resume, and configure music, sound, and vibration.	Should
FR-08	Persist gold and meta upgrades between sessions.	Should
FR-09	Finish a stage with either Win or Game Over.	Must

3. Non-functional Requirements

- Stable performance with large groups of Entity enemies and limited unnecessary GameObject allocation.
- UI layout must be authored as Canvas prefabs instead of being constructed by runtime layout code.
- Gameplay values must be separated from logic through ScriptableObject configuration.
- Visual run speed must follow Entity movement speed to reduce foot sliding.
- Local settings must have safe defaults when saved values do not exist.
- Wave configuration must validate missing IDs, duplicates, and invalid spawn entries.

4. Use Case Diagram

The Use Case diagram is provided as PlantUML source so it can be regenerated or edited without redrawing it. The source is included in Appendix A and delivered as 01_use_case.puml.

5. Use Case Descriptions

UC-01 - Start a Stage

Field	Description
Actor	Player
Precondition	A valid stage and loadout are available.

Field	Description
Main flow	Select stage and weapon, press Start, initialize the stage session, and load GameScene.
Postcondition	The player, HUD, and ECS stage runtime are active.
Exception	Missing or invalid configuration is logged and the invalid entry must not block stage completion permanently.

UC-02 - Move and Auto Attack

Field	Description
Actor	Player
Precondition	The game state is Running.
Main flow	Read movement input, update the player, select a target in range, and fire according to the active weapon profile.
Postcondition	Projectiles are created and damage can be applied.
Exception	Missing or invalid configuration is logged and the invalid entry must not block stage completion permanently.

UC-03 - Spawn an Enemy Wave

Field	Description
Actor	Game System
Precondition	The stage has started or the previous wave is complete.
Main flow	Activate a wave, schedule spawn entries, enqueue requests, and instantiate enemy Entity prefabs.
Postcondition	The configured enemies are active and counted.
Exception	Missing or invalid configuration is logged and the invalid entry must not block stage completion permanently.

UC-04 - Level Up and Choose an Upgrade

Field	Description
Actor	Player
Precondition	The player has collected enough XP.
Main flow	Enter LevelUpState, show upgrade cards, apply the selected upgrade, and resume gameplay.
Postcondition	The chosen statistic or weapon modifier is updated.
Exception	Missing or invalid configuration is logged and the invalid entry must not block stage completion permanently.

UC-05 - Pause and Configure Settings

Field	Description
Actor	Player
Precondition	The application is running.
Main flow	Open Settings, adjust music/sound/vibration, save values, and close or resume.
Postcondition	Settings are applied and stored in PlayerPrefs.
Exception	Missing or invalid configuration is logged and the invalid entry must not block stage completion permanently.

UC-06 - Complete a Stage

Field	Description
Actor	Game System
Precondition	The final wave is terminal and no enemies remain alive.
Main flow	WaveCompletion confirms completion and emits StageCompleted.
Postcondition	The state machine enters WinState.
Exception	Missing or invalid configuration is logged and the invalid entry must not block stage completion permanently.

6. Activity Diagram

The complete gameplay activity flow is supplied as Mermaid source in 02_gameplay_activity.mmd. It covers menu entry, stage initialization, wave activation, combat, wave repetition, Game Over, and Win branches.

III. DETAILED DESIGN

1. UI Design

Screen / Prefab	Main elements	Design responsibility
MainMenuCanvas.prefab	Stage title, upgrade cards, weapon cards, settings, Start	MainMenuManager binds data and events; the prefab owns layout.
GameplayHUDLayout.prefab	HP, XP, level, wave, gold, weapon stats, settings	HUDManager updates values; the Canvas prefab owns layout.
SettingsMenu.prefab	Music slider, Sound slider, vibration, close	Fill and Handle use matching track padding and remain clamped to the slider bounds.
LevelUpMenu.prefab	Upgrade panels	Displays upgrade choices while LevelUpState is active.
Game Over / Win UI	Result and navigation actions	Activated by the game state machine.

The visual system uses a dark blue/purple base with yellow accents. TextMesh Pro provides readable typography. The Start button is visually transparent so only its label remains visible.

2. Hybrid Architecture Diagram

The OOP/ECS boundary and bridge responsibilities are defined in Mermaid source 03_hybrid_architecture.mmd.

3. Class / Component Diagram

The main classes, ECS components, and dependencies are defined in Mermaid class diagram source 04_class_component.mmd.

4. Sequence Diagram

The spawn-to-combat sequence is defined in Mermaid sequence diagram source 05_spawn_combat_sequence.mmd.

5. Gameplay Design

5.1. Wave and Spawn Pipeline

StageConfig contains WaveDefinition records; each wave contains SpawnEntryDefinition records that reference EnemyId values in EnemyCatalog. WaveSpawnAuthoring bakes the authoring configuration into StageRuntime and dynamic buffers. SpawnEntrySchedulerSystem creates SpawnRequest items according to quantity and interval, while SpawnRequestProcessingSystem instantiates the resolved Entity prefab at the requested location. WaveCompletionSystem completes a wave only when all entries are terminal and the alive count reaches zero.

5.2. Weapons and Shotgun Spread

WeaponManager stores damage, projectile count, spread angle, cooldown, range, and projectile speed. PlayerAutoAttackSystem distributes multiple shot directions across the configured cone. The Shotgun therefore fires a pellet group instead of a single projectile. GunStatsSystem combines base values with runtime modifiers from upgrades and meta progression.

5.3. Enemy Visuals and Animation

Mob entities own simulation data such as health, damage, and movement speed. MobVisualBridge creates a matching GameObject model and synchronizes its transform every frame.

MobVisualVariantAuthoring selects the DogMutant presentation for the dog enemy. Animator speed is computed from actual movement speed, run-loop duration, and distance travelled per animation loop, reducing visible foot sliding. Visual scale and ground offset remain configurable independently of the Entity collider.

5.4. State Machine

GameStateMachineRunner coordinates Running, PlayerPaused, LevelUp, GameOver, and Win.

StageCompleted moves the game to WinState; zero player health moves it to GameOverState; an XP threshold temporarily moves it to LevelUpState.

6. Data Model

The project does not use a database server. ScriptableObject assets represent configuration relationships and PlayerPrefs stores local settings and progression. The model is supplied as Mermaid ER source in 06_data_model.mmd.

Entity	Representative attributes	Storage
StageConfig	StageId, DefaultWaveDelay, MaxAliveEnemies, Waves	ScriptableObject
WaveDefinition	WaveId, WaveType, ActivationCondition, WaveDelay, Entries	Nested configuration
SpawnEntryDefinition	EnemyId, Quantity, SpawnInterval, spawn position/area	Nested configuration
EnemyCatalogEntry	EnemyId, EntityPrefab	EnemyCatalog.asset
GunConfig	Damage, cooldown, projectile count, spread, range	ScriptableObject
MobVisualSettings	Prefabs, controllers, loop timing, scale/offset data	Resources asset
PlayerPrefs	Music, sound, vibration, gold, meta upgrades	Local machine

IV. TESTING

The table separates static project inspection from Unity Play Mode verification. “Pass - static” confirms that the required code, prefab, or configuration path exists. “Play Mode required” indicates that runtime behavior still needs final execution evidence on the submission machine.

ID	Area	Action	Expected result	Status
TC-01	Main Menu Canvas	Open MainMenu.	The prefab loads and buttons are bound.	Pass - static
TC-02	Gameplay HUD Canvas	Load GameScene.	HUD values are displayed and layout remains prefab-editable.	Pass - static
TC-03	Settings sliders	Drag Music and Sound to both ends.	Fill follows Handle; Handle stays inside track; volume follows value.	Pass - static
TC-04	Player movement	Use movement controls in Running.	The character moves smoothly in the requested direction.	Play Mode required
TC-05	Wave spawning	Begin a stage.	Correct prefabs, quantity, interval, and alive limit are used.	Pass - static
TC-06	Enemy ground contact	Observe spawned enemies.	Collider and visual offset place the model on the ground.	Play Mode required
TC-07	DogMutant animation	Spawn the dog variant.	Run animation plays and speed follows movement speed.	Pass - static
TC-08	Shotgun spread	Equip and fire the Shotgun.	Multiple projectiles form a configured spread cone.	Pass - static
TC-09	XP and level up	Collect enough XP.	Upgrade selection appears and modifies the chosen stat.	Pass - static
TC-10	Pause and resume	Open Settings/Pause and resume.	Simulation pauses and continues in the correct state.	Play Mode required
TC-11	Win condition	Clear the final wave.	StageCompleted is emitted and WinState is entered.	Pass - static
TC-12	Persistence	Change settings and restart.	PlayerPrefs restore the previous values.	Pass - static

Acceptance Criteria

- No compile errors in the Unity Console.
- Enemy models do not flash at the player position during spawn.
- DogMutant shows no obvious foot sliding at default and modified movement speeds.
- Slider fill, handle, and volume value remain synchronized in the [0,1] range.
- The final wave can complete even when an invalid spawn entry has been marked terminal.

V. TASK ASSIGNMENT

No.	Task	Owner	Status
1	Requirement analysis and architecture design	Vu Viet Tien	Completed
2	OOP/ECS gameplay, waves, combat, and XP	Vu Viet Tien	Completed for the current version
3	Main Menu, HUD, and Settings Canvas	Vu Viet Tien	Completed for the current version
4	Enemy and DogMutant visuals/animation	Vu Viet Tien	Completed for the current version
5	Testing, bug fixing, and report writing	Vu Viet Tien	Final Play Mode regression pending

Self-assessment

The project demonstrates a practical hybrid Unity architecture, data-driven configuration, and editor-friendly Canvas workflows. The highest-priority action before submission is a complete Play Mode regression pass with screenshots and a Unity Profiler capture from a high-enemy-count wave.

VI. INSTALLATION INSTRUCTIONS

1. Prerequisites

Unity Hub, Unity Editor 6000.3.10f1, Windows 10/11, and optionally Git.

2. Open the Project

1. Open Unity Hub and choose Add project from disk.
2. Select the Endless zombie project directory.
3. Select Unity Editor 6000.3.10f1 and allow package restoration to complete.
4. Confirm that the Console contains no compile errors.

3. Run in the Editor

5. Open Assets/Scenes/MainMenu.unity.
6. Press Play.
7. Choose a stage and loadout, then press Tap to Start.
8. Verify the HUD, waves, Shotgun, DogMutant, pause, and settings behavior.

4. Build for Windows

Open File > Build Profiles > Windows, add MainMenu and GameScene to the Scene List, and choose Build. Keep the generated executable and its associated data directory together.

VII. CONCLUSION AND FUTURE DEVELOPMENT

Endless Zombie implements the complete survival-shooter loop: stage entry, wave spawning, automatic combat, XP and upgrades, and win/loss completion. The hybrid architecture provides efficient simulation while preserving a visual Unity workflow for UI, models, animation, and VFX.

- Add bosses and more enemy archetypes with explicit attack state machines.
- Introduce pooling for visual bridges and VFX, then profile CPU time and garbage collection.
- Expand stages, weapon synergies, and meta progression.
- Add automated Play Mode tests for spawning, win conditions, and persistence.
- Improve accessibility with input remapping, contrast options, text scaling, and reduced screen shake.

APPENDIX A. DIAGRAM SOURCE CODE

Use Case is authored in PlantUML. Every other diagram is authored in Mermaid. The same source files are delivered beside this report in the diagrams folder and can be pasted directly into PlantUML or Mermaid-compatible editors.

01_use_case.puml | PlantUML source

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false
skinparam defaultFontName Arial
actor Player
rectangle "Endless Zombie" {
    usecase "Start a Stage" as UC1
    usecase "Move and Aim" as UC2
    usecase "Auto Attack" as UC3
    usecase "Collect XP and Level Up" as UC4
    usecase "Choose an Upgrade" as UC5
    usecase "Pause or Resume" as UC6
    usecase "Adjust Audio and Vibration" as UC7
    usecase "Select Weapon Loadout" as UC8
    usecase "Purchase Meta Upgrades" as UC9
    usecase "Complete or Lose a Stage" as UC10
}
Player --> UC1
Player --> UC2
Player --> UC3
Player --> UC5
Player --> UC6
Player --> UC7
Player --> UC8
Player --> UC9
UC3 .> UC4 : <<include>>
UC4 .> UC5 : <<extend>>
UC1 .> UC10 : <<include>>
@enduml
```

02_gameplay_activity.mmd | Mermaid flowchart source

```

flowchart TD
    A([Open Game]) --> B[Main Menu]
    B --> C[Select stage, weapon, and upgrades]
    C --> D[Load GameScene]
    D --> E[Initialize Player, HUD, and ECS World]
    E --> F[Begin Stage]
    F --> G[Activate Wave]
    G --> H[Schedule Spawn Entries]
    H --> I[Instantiate Enemy Prefabs]
    I --> J[Chase, Auto Attack, Damage, and XP]
    J --> K{Player HP is zero?}
    K -- Yes --> L([Game Over])
    K -- No --> M{Wave cleared?}
    M -- No --> J
    M -- Yes --> N{Final wave?}
    N -- No --> G
    N -- Yes --> O([Win])
  
```

03_hybrid_architecture.mmd | Mermaid flowchart source

```

flowchart LR
    subgraph Presentation[Presentation Layer]
        Canvas[Canvas Prefabs]
        Models[Models and Animators]
        VFX[VFX and Audio]
    end
    subgraph OOP[OOP / GameObject Layer]
        Menu[MainMenuManager]
        State[Game State Machine]
        Input[PlayerInput and CharacterLogic]
        UI[HUDManager and SettingsMenu]
        Meta[MetaProgression and GoldWallet]
    end
    subgraph Bridge[Bridge Layer]
        MV[MobVisualBridge]
        DB[DamageBridge]
        XPB[XPBridge]
        WV[WeaponVfxBridge]
    end
    subgraph ECS[ECS / DOTS Simulation]
        Wave[Wave Progression]
        Spawn[Spawn Scheduler and Processor]
        Mob[Mob Chase and Movement]
        Combat[Projectile and Damage]
        XP[XP and Combat Events]
    end
    Canvas --> UI
    Models --> MV
    VFX --> WV
    Menu --> State
    Input --> Combat
    State --> Wave
    UI <--> Bridge
    Bridge <--> ECS
    Meta --> State
    Wave --> Spawn --> Mob --> Combat --> XP
  
```

04_class_component.mmd | Mermaid classDiagram source

```
classDiagram
    class GameManager {
        +GameStateMachineRunner stateMachine
        +PauseGame()
        +ResumeGame()
    }
    class GameStateMachineRunner {
        +BeginStage()
        +ChangeState(GameStateType)
        +HandleStageCompleted()
    }
    class HUDManager {
        -GameplayHUDView view
        +RefreshHealth()
        +RefreshWaveStatus()
    }
    class WaveSpawnAuthoring {
        -StageConfig stage
        -EnemyCatalog enemyCatalog
        +Bake()
    }
    class StageRuntime {
        +StageId
        +State
        +CurrentWaveIndex
    }
    class WeaponManager {
        +Damage
        +ProjectileCount
        +SpreadAngle
        +AttackCooldown
    }
    class Mob {
        +Health
        +MovementSpeed
        +Damage
    }
    class MobVisualBridge {
        +CreateVisual()
        +SyncTransform()
        +SyncAnimatorSpeed()
    }
    GameManager --> GameStateMachineRunner
    GameStateMachineRunner --> StageRuntime
    HUDManager --> StageRuntime
    WaveSpawnAuthoring --> StageRuntime : bakes
    StageRuntime --> Mob : schedules
    WeaponManager --> Mob : damages
    MobVisualBridge --> Mob : visualizes
```

05_spawn_combat_sequence.mmd | Mermaid sequenceDiagram source

```
sequenceDiagram
    participant GS as GameStateMachine
    participant WP as WaveProgressionSystem
    participant SS as SpawnEntrySchedulerSystem
    participant SP as SpawnRequestProcessingSystem
    participant MS as Mob and Combat Systems
    participant BR as Bridges and HUD
    GS->>WP: BeginStage()
    WP->>SS: Activate current wave
    loop For each spawn entry
        SS->>SP: Enqueue SpawnRequest
        SP->>MS: Instantiate enemy Entity prefab
        MS->>BR: Create and bind visual model
    end
    loop While enemies are alive
        MS->>MS: Chase player and update movement
        MS->>MS: Fire projectiles and apply damage
        MS-->>BR: Damage, VFX, and XP events
    end
    BR-->>WP: Updated alive count
    WP->>WP: Complete wave or activate next wave
    WP-->>GS: StageCompleted
```

06_data_model.mmd | Mermaid erDiagram source

```

erDiagram
    STAGE_CONFIG ||--|{ WAVE_DEFINITION : contains
    WAVE_DEFINITION ||--|{ SPAWN_ENTRY : contains
    SPAWN_ENTRY }o--|| ENEMY_CATALOG_ENTRY : references_by_EnemyId
    ENEMY_CATALOG_ENTRY ||--|| ENTITY_PREFAB : resolves_to
    GUN_CONFIG ||--|| WEAPON_MANAGER : initializes
    MOB_VISUAL_SETTINGS ||--o{ MOB_VISUAL_INSTANCE : creates
    PLAYER_PREFS ||--|| AUDIO_SETTINGS : stores
    PLAYER_PREFS ||--|| GOLD_WALLET : stores
    PLAYER_PREFS ||--|| META_PROGRESSION : stores

    STAGE_CONFIG {
        string StageId
        float DefaultWaveDelay
        int MaxAliveEnemies
    }
    WAVE_DEFINITION {
        string WaveId
        string WaveType
        float WaveDelay
    }
    SPAWN_ENTRY {
        string EnemyId
        int Quantity
        float SpawnInterval
    }
    ENEMY_CATALOG_ENTRY {
        string EnemyId
        EntityPrefab Prefab
    }
  
```

REFERENCES

Unity Technologies. Unity Manual 6000.x.

Unity Technologies. Entities 1.4 documentation.

Unity Technologies. Universal Render Pipeline 17 documentation.

Endless Zombie source code, prefabs, and ScriptableObject assets, reviewed August 2026.

Project2_GD1305_VuVietTien.pdf, used only as the report-structure reference.